

Project Report NNGraph DSL Compiler

Intro:

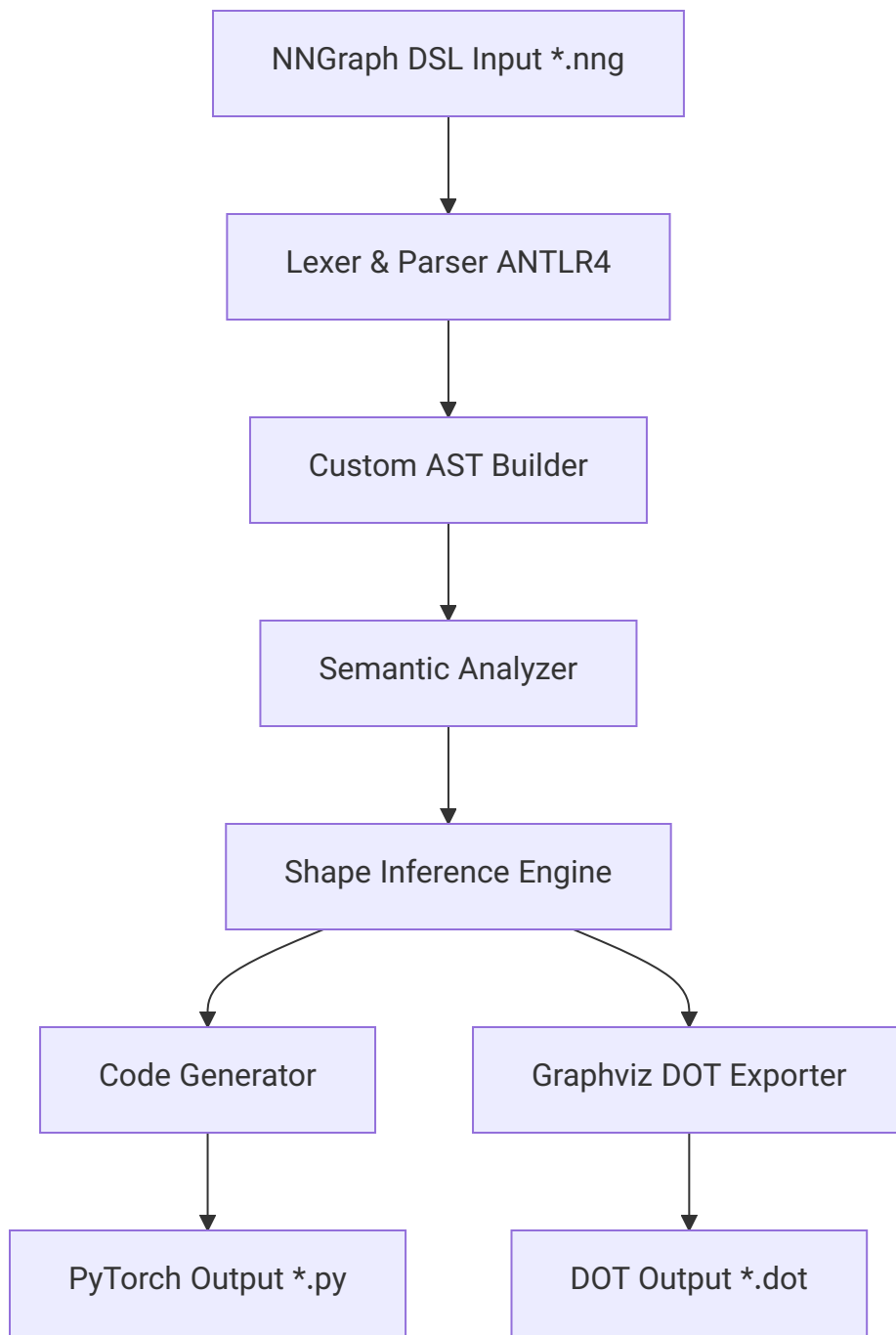
This report provides a comprehensive, end-to-end explanation of the **NNGraph DSL Compiler** project. The compiler is designed to translate a declarative, domain-specific language (DSL) for neural network computational graphs—referred to as the **NNGraph DSL**—into executable and fully optimized PyTorch code.

1. Introduction & Overview

The primary goal of the NNGraph DSL is to simplify the specification of deep learning architectures by decoupling structural network topology from mathematical dimension tracking. Instead of manually writing error-prone PyTorch constructor and forwarding code, users specify the computational graph declaratively. The compiler automatically validates the graph, infers missing tensor dimensions, resolves execution ordering, and produces a ready-to-use PyTorch `nn.Module` subclass.

The Compiler Pipeline

The compiler is organized as a classic pipeline comprising five main phases:



1. **Lexical and Syntactic Analysis (ANTLR4):** Consumes the raw source code (`.nng`) and structures it into a parse tree based on the grammar defined in [nngraph.g4](#).
2. **AST Construction:** An ANTLR listener walks the parse tree and translates it into a simplified, custom Abstract Syntax Tree (AST) using [CustomListener](#) and [make_ast_subtree](#).
3. **Semantic Analysis:** Implemented in [semantic_analyzer.py](#). It builds a target model representation, runs critical static checks (structural validations, cycle detection, parameters, and reachability), and yields warning/error diagnostics.
4. **Shape Inference Engine:** Implemented in [shape_inference.py](#). It propagates shape dimensions through graph edges starting from the inputs, dynamically computing spatial

size, channels, and feature counts. It automatically infers missing parameters (e.g. `in_features` for `Linear` layers, `in_ch` for `Conv2d`, etc.).

5. **Code Generation:** Implemented in `code_generator.py`. It formats the model properties into a clean PyTorch Python module (`class ModelName(nn.Module)`), topological execution blocks, variable branch names, and residual shortcuts.
 6. **Graphviz DOT Exporter:** Implemented in `graph_export.py`. It converts the compiled model representation into Graphviz's DOT format for easy visual debugging and documentation.
-

2. Architecture & Modules

The codebase is organized into modular files with clean interfaces. Below is an architectural overview of each component:

Lexer & Parser Grammar

- **File:** `nngraph.g4`
- **Description:** Contains the lexer tokens and parser grammar rules. The grammar specifies three optional/mandatory blocks:
- `modelDecl` : Declares the model name and input/output tensors.
- `graphDecl` : Defines the network nodes (with parameter value assignments) and directed edges.
- `configDecl` : Provides optional execution parameters like `batch_size` and `device`.

AST Generation

- **Files:** `custom_listener.py` and `required_code_collection/make_ast_subtree.py`
- **Description:** Inherits from `nngraphListener` to execute custom AST construction actions upon exiting grammar rules. The helper `make_ast_subtree` processes context children, filters out structural punctuation (braces, parentheses, commas, arrows), and binds tree nodes to form a clean AST represented by the `AST` class.

Semantic Analyzer

- **File:** `semantic_analyzer.py`
- **Description:** Implements the `SemanticAnalyzer` class, which walks the AST and creates the internal `SemanticModel` representation. It implements several validation passes:
- **Edge Wiring and Undefined Node Validation:** Links edges and ensures all source/destination targets exist.
- **Parameter Verification:** Checks parameters against `LAYER_SPECS` for name spelling and value types.

- **Arity and Configuration Verification:** Ensures nodes like `Residual` have exactly two incoming edges, and `Add` or `Concat` have at least two.
- **Cycle Detection:** Validates that the graph is a Directed Acyclic Graph (DAG) using Kahn's topological sorting algorithm.
- **Reachability Verification:** Checks if all nodes are reachable from the main input, and that the designated output is reachable from the input.
- **Orphan Node Identification:** Warns the user about nodes that are declared but never connected in the computational graph.

Shape & Dimension Inference

- **File:** `shape_inference.py`
- **Description:** Implements `infer_missing_dims`. It propagates shapes recursively through a memoized depth-first search. It handles dimension logic and calculations for layers like `Linear`, `Conv1d/2d`, `MaxPool2d`, `BatchNorm2d`, `Flatten`, `LayerNorm`, `Concat`, and `Split`. Missing input dimensions are computed based on the output shapes of preceding layers.

Code Generator

- **File:** `code_generator.py`
- **Description:** Implements `CodeGenerator`. It takes the validated `SemanticModel` and outputs target PyTorch code. In the forward pass, it writes variable assignments in topological order. It supports branching variables (generating safe names like `a`, `b`, etc. when a node's output splits) and residual path snapshotting.

Graph Export Utility

- **File:** `graph_export.py`
- **Description:** Implements `export_dot`, which translates the `SemanticModel` to Graphviz DOT notation. It highlights inferred attributes (with an asterisk `*`), input nodes (light blue ellipse), output nodes (double border and light orange), and intermediate layer boxes.

3. Code Explanations & Snippets

ANTLR4 Grammar Design (`nngraph.g4`)

The compiler uses ANTLR4 to define both lexer tokens and parser structures. Below is a key section of the grammar file showing how nodes and edges are parsed:

```
// nngraph.g4: Parser rules for graph definitions and parameters

graphDecl : GRAPH LBRACE graphItem* RBRACE ;

graphItem: nodeDecl | edgeDecl ;

nodeDecl : NODE IDENTIFIER COLON layerType LPAREN paramList? RPAREN ;

layerType : IDENTIFIER ;

edgeDecl : EDGE IDENTIFIER ARROW IDENTIFIER edgeLabel? ;

edgeLabel : LBRACK IDENTIFIER EQ STRING RBRACK;

paramList : param (COMMA param)* COMMA? ;

param : IDENTIFIER EQ value ;

value : number | BOOL | STRING | shape | NONE ;
```

Explanation:

- `graphDecl` defines a block enclosed in braces (`{ }`) that lists nodes and edges.
- `nodeDecl` parses layer instantiations like `node fc1 : Linear(out_features=256)`.
- `edgeDecl` parses connection declarations like `edge fc1 -> relu1` and supports optional label tags like `[label="shortcut"]` to document or specify weights/branches.

- `param` maps identifiers to literal values (numbers, booleans, strings, tuple-shapes, or `None`).

AST Builder (Listener-based Construction)

In `custom_listener.py`, AST subtrees are assembled when the listener leaves parsing rules:

```
def exitNodeDecl(self, ctx):  
  
    self._keep_token(ctx.IDENTIFIER())  
  
    # children become: id, layerType, (params?)  
  
    make_ast_subtree(self.ast, ctx, "node", keep_node=True)  
  
def exitEdgeDecl(self, ctx):  
  
    self._keep_token(ctx.IDENTIFIER(0)) # src  
  
    self._keep_token(ctx.IDENTIFIER(1)) # dst  
  
    make_ast_subtree(self.ast, ctx, "edge", keep_node=True)
```

Explanation:

- `exitNodeDecl` processes a node declaration context. It flags the node identifier token to be kept in the tree, and then invokes `make_ast_subtree` to group the node ID, layer type, and its parameter list under a new AST subtree labeled `"node"`.
- `exitEdgeDecl` extracts the source and destination identifiers and packs them under an AST subtree labeled `"edge"`.

Semantic Analyzer: Kahn's Algorithm for Topological Sort & Cycle Detection

To generate sequentially executable code and ensure the model has no circular dependencies, the compiler uses Kahn's algorithm inside `semantic_analyzer.py`:

```

def _check_acyclic_and_topo(self, model, valid_ids):

    # Kahn's algorithm over input + graph nodes

    indegree = {nid: 0 for nid in valid_ids}

    adj = defaultdict(list)

    for src, dst in model.edges:

        if src in indegree and dst in indegree:

            adj[src].append(dst)

            indegree[dst] += 1

    queue = deque(sorted(n for n, d in indegree.items() if d == 0))

    order = []

    while queue:

        cur = queue.popleft()

        order.append(cur)

        for nxt in adj[cur]:

            indegree[nxt] -= 1

            if indegree[nxt] == 0:

                queue.append(nxt)

    if len(order) != len(indegree):

        remaining = [n for n in indegree if n not in set(order)]

        self.diag.error("Cycle detected involving nodes: " + ",
            ".join(sorted(remaining)) + ".")

```

```
hint="NNGraph graphs must be acyclic DAGs.")

else:

    model.topo_order = order
```

Explanation:

- **Step 1:** It calculates the in-degree of all nodes and populates an adjacency list `adj`.
- **Step 2:** All nodes with an in-degree of 0 (starting with the model input) are pushed to a processing queue.
- **Step 3:** It pulls nodes from the queue one by one, appends them to the topological `order`, and decrements the in-degree of their neighbors. If a neighbor's in-degree drops to 0, it gets queued.
- **Step 4:** If the length of the topological order does not match the total number of valid nodes, there is a cycle, and the analyzer records a compilation error naming the offending cycle components.

Code Generator: Dynamic Variable Routing

When a node's output flows into multiple subsequent layers, the code generator must manage intermediate variables to prevent overwriting values. This is managed in `code_generator.py`:

```
def _needs_fresh_names(self, model, source_id):

    out_edges = self._out_edges_of.get(source_id, [])

    if len(out_edges) <= 1:

        return False

    real_count = 0

    for dst in out_edges:

        in_deg = len(model.nodes[dst].in_edges)

        if in_deg <= 1:

            real_count += 1
```

```

continue

is_covered_residual_shortcut = ( model.nodes[dst].layer_type == "Residual"
and self._residual_shortcut_of.get(source_id) == dst)

if not is_covered_residual_shortcut:

return True

return real_count >= 2


def _next_letter(self):

idx = self._letter_idx

self._letter_idx += 1

return chr(ord('a') + idx) if idx < 26 else f"br{idx}"

```

Explanation:

- `_needs_fresh_names` checks if the source node's outputs diverge into multiple paths that require unique variable names. If a branch is part of a standard residual connection, the compiler handles it as a shortcut state snapshot and does not allocate a new arbitrary branch name.
- `_next_letter` assigns letters like `a`, `b`, `c`, etc. for branch variables sequentially to write clean, human-readable forward statements (e.g. `a = self.fc1(x)`).

4. Extra Features Implementation

Beyond the base requirements of parsing and compiling node-edge connections, the NNGraph DSL Compiler implements two advanced features:

A. Trivial Tensor Dimension Inference

Writing neural network graphs is often repetitive because output sizes must match subsequent input sizes. For example, a `Linear` layer needs to match the preceding layer's output dimensions, and a 2D convolution requires matching channels.

The NNGraph compiler automates this task using the **Tensor Dimension Inference Engine** in `shape_inference.py`.

The Core Shape Engine

The engine defines a main recursive solver `get_shape(node_id, visiting)` to trace shapes back to the model input node. Each layer type has a shape handler:

```
def h_linear(node, shapes):  
  
    shape = merge_shapes(node, shapes)  
  
    fill_or_check(node, "in_features", shape[-1] if shape else None)  
  
    out_f = get_int(node, "out_features")  
  
    if shape is None or out_f is None:  
  
        return None  
  
    return shape[:-1] + (out_f,)
```

```
def fill_or_check(node, pname, computed, as_shape=False):  
  
    if computed is None:  
  
        return  
  
    lit = node.params.get(pname)  
  
    if lit is None:  
  
        value = (computed,) if as_shape else computed  
  
        node.params[pname] = Literal(SHAPE if as_shape else INT, value,  
                                     INFERRED_MARK)  
  
        inferred.append((node.id, pname, computed))  
  
        return  
  
    declared = lit.value[-1] if lit.type == SHAPE else lit.value
```

```

if isinstance(declared, int) and declared != computed:

diag.error(

f"Node '{node.id}' ({node.layer_type}) declares "

f"'{pname}={declared}', but its predecessor(s) produce {computed}.",

hint="Fix the declared parameter or the graph wiring.")

```

How Dimension Inference Works:

1. When the compiler encounters a `Linear` layer, it retrieves the output shape of its predecessor via `shapes`.
2. The final dimension of the predecessor's shape (the feature count) is extracted.
3. `fill_or_check` checks if `in_features` was declared by the user:
 - **Case A: Parameter is missing:** The compiler automatically constructs a new `Literal` containing the computed dimension, flags it with the tag "`<inferred>`", and stores it.
 - **Case B: Parameter is present:** The compiler verifies if the user-declared value matches the computed incoming dimension. If they mismatch, it reports a compilation error.
4. The output shape is updated to reflect the new feature dimension `out_features`, which propagates to subsequent layers.

Similar inference handlers are defined for:

- `Conv2d`: Computes output spatial shapes using:

$$\text{out} = \left\lfloor \frac{\text{size} + 2 \times \text{padding} - \text{kernel}}{\text{stride}} \right\rfloor + 1$$

It also automatically infers the input channels (`in_ch`) from the predecessor's channel count.

- `BatchNorm2d`: Inferred `num_features` is set to match the incoming channel size.
- `LayerNorm`: Inferred `normalized_shape` is set to match the final dimension of the incoming shape.
- `Flatten`: Inferred output dimension is set to the product of all dimensions between `start_dim` and `end_dim`.
- `Concat`: Merges shapes along a concatenation axis, validating that all other dimensions match.

B. Graphviz DOT Export

To inspect compiled computational graphs visually, the utility in `graph_export.py` formats the parsed model into Graphviz DOT notation.

```
def export_dot(model) -> str:

    lines = [

        f'digraph "{model.name or "NNGraph"}" {{',

        '    rankdir=LR;',

        '    node [shape=box, fontname="Helvetica", style=filled,'
        fillcolor="#f5f5f5"];',

        '',

    ]

    if model.input_id:

        shape_str = ", ".join(str(d) for d in model.input_shape)

        lines.append(

            f' {model.input_id} [shape=ellipse, fillcolor="#cfe8ff", '

            f'label="{_escape(model.input_id)}\\ntensor({shape_str})";'

        )

    for node in model.nodes.values():

        label = f"{_escape(node.id)}\\n{_escape(node.layer_type)}"
        ({_escape(_format_params(node))})"

        attrs = f'label="{label}"'

        if node.id == model.output_id:
```

```

attrs += ', peripheries=2, fillcolor="#ffe9b3"'

lines.append(f' {node.id} [{attrs}];')

lines.append('')

for src, dst in model.edges:

    edge_label = model.edge_labels.get((src, dst))

    if edge_label:

        lines.append(f' {src} -> {dst} [label="{_escape(edge_label)}"];;')

    else:

        lines.append(f' {src} -> {dst};')

lines.append('}')

return "\n".join(lines) + "\n"

```

Formatting & Visualization Logic:

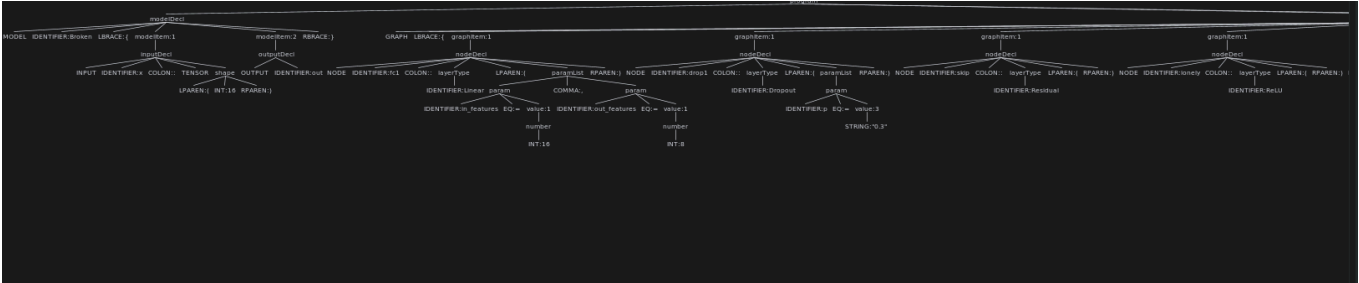
- **Layout:** Graphs are oriented left-to-right (`rankdir=LR`) to match standard neural network diagrams.
- **Input Node:** Colored blue (`#cfe8ff`) with an `ellipse` shape, displaying its name and shape tuple (e.g. `x\ntensor(784)`).
- **Intermediate Nodes:** Plotted as light-gray boxes (`#f5f5f5`). Labels display both the layer type and its parameters.
- **Output Node:** Colored orange (`#ffe9b3`) and decorated with a double border (`peripheries=2`).
- **Inferred Parameter Highlights:** Parameters that were automatically inferred by the shape engine have an asterisk (`*`) appended to their printed values (e.g., `in_features=784*`), indicating they were resolved by the compiler rather than specified by the user.

5. Visual Results:

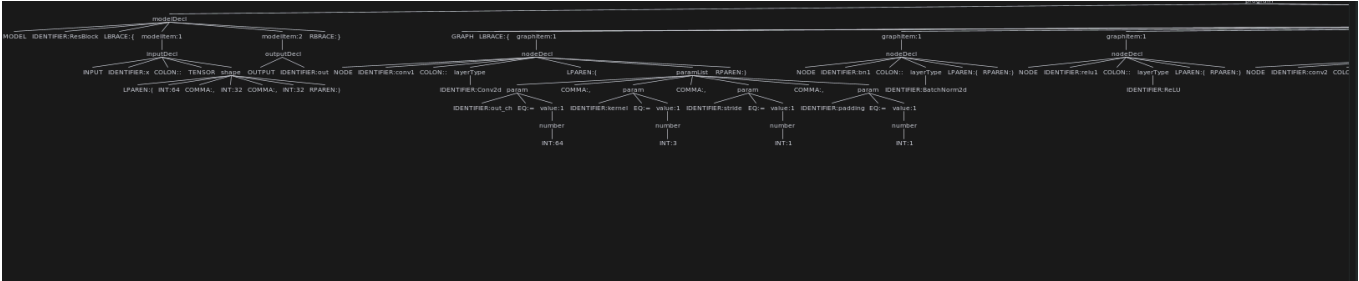
ANTLR Parse Tree

This parse tree visualizes the syntax structure produced by the ANTLR lexer and parser before the AST builder simplifies it:

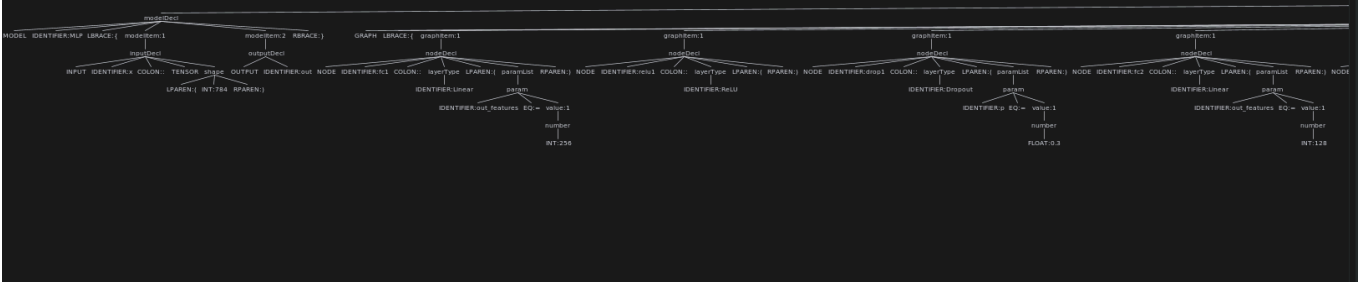
bad.ng:



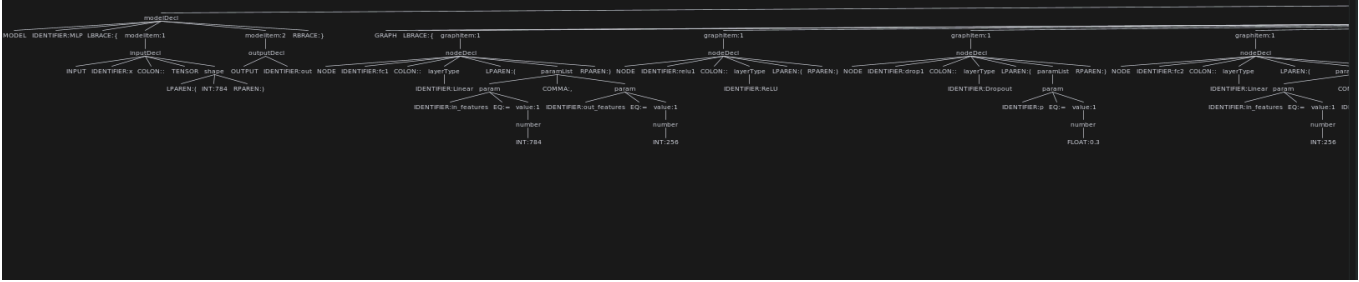
cnn_inferred.ng:



mlp_inferred.ng:

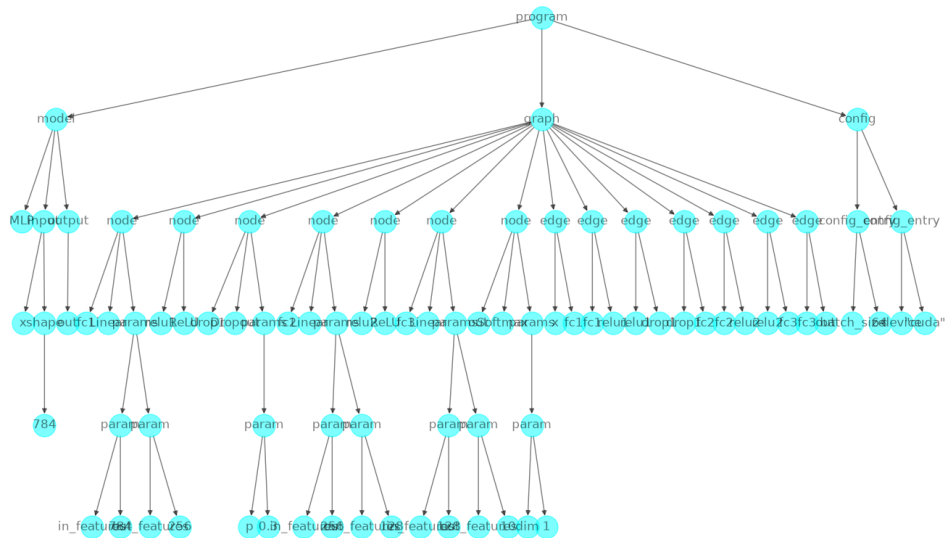


mlp.ng:



Computational Graph Visualization: MLP Classifier

This diagram represents the computational graph of the Multi-Layer Perceptron (MLP) defined in `mlp_inferred.nng`. Notice how the `in_features` parameters are automatically inferred:



```
Semantic analysis passed for model 'MLP' (0 warning(s)).
Generated PyTorch code written to 'output/mlp.py'.
Graph diagram (Graphviz DOT) written to 'output/mlp.dot'.
```

```
1 digraph "MLP" {
2     rankdir=LR;
3     node [shape=box, fontname="Helvetica", style=filled, fillcolor="#f5f5f5"];
4
5     x [shape=ellipse, fillcolor="#cfe8ff", label="x\ntensor(784)"];
6     fc1 [label="fc1\nLinear(in_features=784, out_features=256)"];
7     relu1 [label="relu1\nReLU()"];
8     drop1 [label="drop1\nDropout(p=0.3)"];
9     fc2 [label="fc2\nLinear(in_features=256, out_features=128)"];
10    relu2 [label="relu2\nReLU()"];
11    fc3 [label="fc3\nLinear(in_features=128, out_features=10)"];
12    out [label="out\nSoftmax(dim=1)", peripheries=2, fillcolor="#ffe9b3"];
13
14    x -> fc1;
15    fc1 -> relu1;
16    relu1 -> drop1;
17    drop1 -> fc2;
18    fc2 -> relu2;
19    relu2 -> fc3;
20    fc3 -> out;
21 }
22
```

```

1 import torch
2 import torch.nn as nn
3
4
5 class MLP(nn.Module): 2+ usages
6     def __init__(self):
7         super(MLP, self).__init__()
8         self.fc1 = nn.Linear(in_features=784, out_features=256)
9         self.relu1 = nn.ReLU()
10        self.drop1 = nn.Dropout(p=0.3)
11        self.fc2 = nn.Linear(in_features=256, out_features=128)
12        self.relu2 = nn.ReLU()
13        self.fc3 = nn.Linear(in_features=128, out_features=10)
14        self.out = nn.Softmax(dim=1)
15
16    def forward(self, x):
17        x = self.fc1(x)
18        x = self.relu1(x)
19        x = self.drop1(x)
20        x = self.fc2(x)
21        x = self.relu2(x)
22        x = self.fc3(x)
23        x = self.out(x)
24        return x
25
26 if __name__ == '__main__':
27     device = torch.device('cuda')
28     model = MLP().to(device)
29     x = torch.randn(64, 784).to(device)
30     print(model(x).shape)
31

```

Computational Graph Visualization: CNN ResBlock

This diagram represents a complex convolutional block with branching paths and a residual merge defined in `cnn_inferred.nng`:

```

1 digraph "ResBlock" {
2     rankdir=LR;
3     node [shape=box, fontname="Helvetica", style=filled, fillcolor="#f5f5f5"];
4
5     x [shape=ellipse, fillcolor="#cfe8ff", label="x\ntensor(64, 32, 32)"];
6     conv1 [label="conv1\nConv2d(in_ch=64*, out_ch=64, kernel=3, stride=1, padding=1)"];
7     bn1 [label="bn1\nBatchNorm2d(num_features=64*)"];
8     relu1 [label="relu1\nReLU()"];
9     conv2 [label="conv2\nConv2d(in_ch=64*, out_ch=64, kernel=3, stride=1, padding=1)"];
10    bn2 [label="bn2\nBatchNorm2d(num_features=64*)"];
11    skip [label="skip\nResidual()"];
12    relu2 [label="relu2\nReLU()"];
13    out [label="out\nFlatten()", peripheries=2, fillcolor="#ffe9b3"];
14
15    x -> conv1;
16    conv1 -> bn1;
17    bn1 -> relu1;
18    relu1 -> conv2;
19    conv2 -> bn2;
20    bn2 -> skip;
21    x -> skip [label="shortcut"];
22    skip -> relu2;
23    relu2 -> out;
24 }
25

```

```
1 import torch
2 import torch.nn as nn
3
4
5 class ResBlock(nn.Module): 1 usage
6     def __init__(self):
7         super(ResBlock, self).__init__()
8         self.conv1 = nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=1)
9         self.bn1 = nn.BatchNorm2d(64)
10        self.relu1 = nn.ReLU()
11        self.conv2 = nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=1)
12        self.bn2 = nn.BatchNorm2d(64)
13        self.relu2 = nn.ReLU()
14        self.out = nn.Flatten()
15
16    def forward(self, x):
17        identity = x
18        x = self.conv1(x)
19        x = self.bn1(x)
20        x = self.relu1(x)
21        x = self.conv2(x)
22        x = self.bn2(x)
23        x = x + identity
24        x = self.relu2(x)
25        x = self.out(x)
26        return x
27
```